

Настройка окружения для разработки

Содержание

Основной вариант: C/C++, GLFW + GLEW.....	1
GNU / Linux.....	2
Установка необходимых пакетов [deb-based].....	2
Установка необходимых пакетов [rpm-based].....	2
Установка необходимых пакетов [прочие].....	2
Компиляция.....	2
MacOS.....	2
Установка необходимого софта.....	2
Компиляция.....	2
Windows.....	3
Шаг 0: проверка поддержки OpenGL.....	3
Шаг 1: установка MS VS.....	3
Шаг 2: загрузка GLEW.....	3
Шаг 3: загрузка GLFW.....	3
Шаг 4: создание проекта.....	3
Шаг 5: задание каталогов библиотек.....	4
Шаг 6: подключение библиотек к проекту.....	4
Шаг 7: сборка проекта.....	4
Шаг 8: добавление DLL-файла GLEW.....	4
Постскриптум.....	5
Итоги.....	5
Альтернативные варианты.....	5
C/C++.....	5
GLAD.....	5
SDL.....	6
Другие языки программирования.....	6
Java.....	6
C#.....	6
Python.....	6
Rust.....	6
Go.....	7
JavaScript.....	7
Приложение А. Исходный код программы.....	7
Приложение Б. Исходный код отладочной функции.....	9

Основной вариант: C/C++, GLFW + GLEW

Стандарт OpenGL описывает только и исключительно работу с 3D-графикой, причем делает это с помощью набора расширений, которые группируются в «мажорные» релизы стандарта. Поэтому в дополнение к библиотеке, реализующей стандарт OpenGL (которая у вас на компьютере уже, скорее всего, есть, ибо она входит в состав драйвера видеокарты) требуется еще две — для загрузки расширений и для взаимодействия с оконной подсистемой ОС. Вам предлагается использовать для этих целей библиотеки GLEW (GL Extension Wrapper) и GLFW (GL FrameWork).

Концептуально ничего сложного нет - нужно скачать двоичные и заголовочные файлы библиотек, а также установить и настроить инструменты разработчика для языков C/C++.

Скелет кода программы для старта находится в приложении А. В пособии он описан подробно. Программа должна всего лишь создать окно и закрасить его синим цветом.

В приложении Б приведен код функции, которая может быть полезна для отладки программ, использующих API OpenGL. Однако имейте в виду, что она требует поддержки либо расширения AMD_debug_output / ARB_debug_output / KHR_debug, либо OpenGL 4.3, в который это расширение вошло как обязательное.

GNU / Linux

Обратите внимание, что в разных дистрибутивах имена пакетов и библиотек могут незначительно отличаться. Делайте на это поправку!

Установка необходимых пакетов [deb-based]

Если у вас Debian, Ubuntu, Linux Mint или что-то из производных — вам сюда.

```
sudo apt install mesa-common-dev libglfw3-dev libglew-dev build-essential
```

Установка необходимых пакетов [rpm-based]

Если у вас Fedora, CentOS, RHEL или что-то из производных — вам сюда.

```
sudo dnf install mesa-libGL-devel glfw-devel glew-devel gcc-g++ make
```

Установка необходимых пакетов [прочие]

Если у вас Gentoo / Arch Linux, с вероятностью в 99% все нужное у вас уже стоит. Если вдруг нет или дистрибутив иной — пишите, разберемся :)

Компиляция

Чтобы собрать программу, достаточно выполнить следующую команду:

```
g++ main.cpp -lGL -lGLEW -lglfw -lX11 -o main
```

Еще раз обратите внимание на замечание об именах библиотек! Скажем, вместо флага -lglfw может потребоваться -lglfw3.

MacOS

С MacOS все лишь немного сложнее, чем с GNU/Linux.

Установка необходимого софта

1. Поставить Xcode из AppStore.
2. Поставить менеджер пакетов Homebrew (требует установленного Xcode): <https://brew.sh/ru/>
3. Установить библиотеки командой `brew install glfw3 glew`

Компиляция

Обратите внимание на флаги компиляции, и — особенно — на пути до каталогов библиотек! Они зависят от установленных у вас версий библиотек. Их можно получить из команды `brew list --versions`

```
g++ main.cpp -framework OpenGL -lGLEW -lglfw3 -I/usr/local/include  
-L/usr/local/Cellar/glew/2.3.1/lib/ -L/usr/local/Cellar/glfw3/3.4/lib/ -o  
main.app
```

Windows

С виндой все, как обычно, сложно...

Настоятельно предлагается прочитать инструкцию до конца прежде, чем ее выполнять.

Шаг 0: проверка поддержки OpenGL

Для начала убедимся, что машина вообще тянет OpenGL (а то ведь мало ли...). Самый простой вариант — зайти в диспетчер устройств и посмотреть, как определяется видеоадаптер. Если там указано что-то **кроме** «Базовый видеоадаптер Microsoft», то с вероятностью 99% все в порядке. Для 100% гарантии можете скачать утилиту <https://www.techpowerup.com/download/techpowerup-gpu-z/>, запустить и посмотреть в правый нижний угол. Если там есть галочка рядом с OpenGL и цифра выше 3.0 — все хорошо. Если галочки нет, но чуть выше в поле DirectX Support указано хотя бы 10 — то все тоже хорошо. Если нет... Пишите, разберемся.

Ну и если у вас идут хоть какие-то относительно современные 3D-игры (да тот же Minecraft 1.18+), то проблем быть не должно :)

Шаг 1: установка MS VS

Для начала необходимо установить Microsoft Visual Studio. Именно полноценную Visual Studio, не VS Code! При установке обратите внимание на то, чтобы были выбраны инструменты для разработки на C/C++ (иногда они бывают по умолчанию не отмечены).

Конкретная версия MS VS значения не имеет, но рекомендуется выбрать 2017, 2019 или 2022. На 2013 и 2015 все тоже можно настроить, но потребуются дополнительные костыли. 2026 можете попробовать на свой страх и риск, мне не доводилось с ней работать.

Шаг 2: загрузка GLEW

Качаем GLEW: <https://github.com/nigels-com/glew/releases/download/glew-2.3.1/glew-2.3.1-win32.zip>

Разархивируем куда-нибудь, например, в специально созданную поддиректорию OpenGL в корне диска C:. Итоговый путь получится C:\OpenGL\glew-2.3.1.

Шаг 3: загрузка GLFW

Качаем GLFW: <https://github.com/glfw/glfw/releases/download/3.4/glfw-3.4.bin.WIN64.zip>

Также разархивируем, для определённости — в ту же директорию OpenGL.

Шаг 4: создание проекта

Создаём консольное приложение C++ в MS VS. В файл `main.cpp` вставляем содержимое приложения А.

Если у вас версия MS VS 2013 или 2015, при создании проекта снимите галочку «Use precompiled headers» / «Использовать прекомпилированные заголовки», а после создания — удалите файлы `stdafx.cpp` и `stdafx.h`.

Не забудьте на панели отладки переключить архитектуру проекта с «x86» (по умолчанию) на «x64»!

Шаг 5: задание каталогов библиотек

- Идём в «Solution Explorer» / «Обозреватель решения». Кликаем правой кнопкой на проекте и выбираем «Properties» / «Свойства».
- Идём в раздел «VC++ Directories» / «Каталоги VC++».

- В «Include Directories» / «Каталоги включаемых файлов» добавляем пути:

```
C:\OpenGL\glfw-3.4.bin.WIN64\include  
C:\OpenGL\glew-2.3.1\include
```

- В «Library Directories» / «Каталоги библиотек» добавляем пути:

```
C:\OpenGL\glfw-3.4.bin.WIN64\lib-vc2017  
C:\OpenGL\glew-2.3.1\lib\Release\x64
```

Обратите внимание, что путь для библиотеки GLFW указывается с учетом версии MS VS. В примере указана 2017, скорректируйте с учетом своей. Если у вас 2013 — попробуйте указать каталог 2015. Если 2026 — каталог 2022. Также поправьте версии библиотек и / или пути к каталогам, если они у вас отличаются.

Шаг 6: подключение библиотек к проекту

- Не закрывая окно свойств проекта, раскрываем раздел «Linker» / «Компоновщик» и переходим в подраздел «Input» / «Ввод».
- Добавляем в пункт «Additional Dependencies» / «Дополнительные зависимости» следующие библиотеки:

```
opengl32.lib  
glfw3.lib  
glew32.lib
```

Шаг 7: сборка проекта

Собираем проект. По идее, всё должно скомпилироваться и слинковаться (но не запускаться).

Шаг 8: добавление DLL-файла GLEW

Наша программа использует библиотеки OpenGL, GLFW и GLEW:

- Библиотека OpenGL является частью драйвера видеоадаптера и установлена в системе.

- Для библиотеки GLFW мы выбрали статический вариант компоновки, т. е. она станет частью нашего приложения при его сборке.
- А вот для GLEW такого варианта, к сожалению, нет.

Поэтому нам необходимо настроить MS VS так, чтобы она при сборке проекта автоматически копировала DLL-файл библиотеки GLEW в каталог, где его сможет найти наше приложение. Можно, конечно, руками ее копировать каждый раз при пересборке проекта... Но зачем?

- В окне «Solution Explorer» / «Обозреватель решения» тыкаем правой кнопкой на проекте и выбираем «Add» => «Existing Item...» / «Добавить» => «Существующий элемент...».
- Находим библиотеку glew32.dll (она должна лежать в C:\OpenGL\glew-2.3.1\bin\Release\x64).
- После добавления тыкаем на нее в списке файлов проекта и открываем свойства.
- В свойствах нужно задать «Element Type» / «Тип элемента» как «Copy File» / «Копирование файла».

Если у вас MS VS 2013 или 2015, то такой опции в свойствах файла **не будет**. Тем не менее, система сборки ее поддерживает; для ее включения нужно вручную в Блокноте отредактировать файл проекта, прописав необходимые строки. Чтобы не загромождать и без того огромную инструкцию, вам предлагается найти их в интернете самостоятельно :)

Постскриптум

В Windows нынче есть модный NuGet. Я с ним не работал, но студенты активно ставили всё необходимое из него и говорили, что это проще и удобнее, чем то, что расписано тут. Соответственно, можете попробовать. Хотя ручная настройка полезна просто для понимания того, как оно все устроено и взаимодействует.

Итоги

При условии поддержки OpenGL 3.3 и выше программа должна собраться, запуститься и показать пустое окно с синим фоном.

Если 3.3 не поддерживается — попробуйте запросить версию 2.1.

Для этого надо изменить код в шаблоне вот так:

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 2);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 1);
//glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
//glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
```

Обращаю внимание, что вторые две строчки *закомментированы*.

После этого окно-то должно создаться, но вот шейдеры, которые мы будем писать в дальнейшем, компилироваться не будут, так как не поддерживается 330-й GLSL. Их нужно будет модифицировать.

Альтернативные варианты

C/C++

GLAD

Можно заменить библиотеку GLEW на GLAD: <https://github.com/Dav1dde/glad>. Она ничем не лучше и не хуже, просто «другая». Пример ее использования есть в README. По сути, заменить придется только заголовочный файл и строки инициализации. Из плюсов — это header-only библиотека: достаточно одного заголовочного файла. Из минусов — нужно этот файл получить из конфигулятора, корректно выбрав необходимые опции :)

SDL

Всю связку GLFW + GLEW можно полностью заменить на библиотеку SDL: <https://libsdl.org/>. В отличие от предложенной для лаб пары, это полнофункциональное решение для разработки компьютерных игр «с нуля», без использования готового движка. SDL поддерживает модули для работы с контроллерами, звуком, изображением, текстом и шрифтами, сетью и много чего еще. На ее основе созданы такие игры, как Counter Strike: Global Offensive, Factorio, Team Fortress 2, Dwarf Fortress; кроме того, SDL в качестве базы используют игровые движки Source, Unreal и Godot. Шаблон при ее использовании придется частично переписать, но существенно свою структуру он не поменяет.

Другие языки программирования

В целом, использование других ЯП для выполнения лабораторных не запрещается; единственное условие — использование максимально «тонких» библиотек-оберток поверх OpenGL (т. е. без автоматической работы с памятью, упрощения API и т. п.). Далее будут приведены библиотеки для нескольких популярных ЯП, которые без проблем можно использовать. Ориентируйтесь на них.

Java

Для OpenGL в Java традиционно используется LWJGL: <https://www.lwjgl.org/>. На ее основе построен, например, Minecraft. В пару к ней для математики (вектора, матрицы) можно взять JOML: <https://github.com/JOML-CI/JOML>

C#

Для C# рекомендуется использовать проверенную временем OpenTK: <https://opentk.net/>. Математика уже идет «в комплекте».

Также есть более современная, модная, молодежная Silk.NET: <https://github.com/dotnet/Silk.NET>. Математика, опять же, «в комплекте».

Python

В случае Python предлагается задействовать PyOpenGL: <https://github.com/mcfletch/pyopengl>. Для математики можно использовать старый добрый NumPy, либо познакомиться с PyGLM: <https://github.com/Zuzu-Typ/PyGLM>

Rust

Для Rust возможность загрузки функций OpenGL предоставляет крейт `gl-rs`:

<https://github.com/rust-windowing/gl-rs>. Дополнительно к нему потребуется библиотека для работы с окнами — `glutin`: <https://github.com/rust-windowing/glutin>. В качестве математической библиотеки можно взять <https://github.com/dimforge/nalgebra> или <https://github.com/bitshifter/glam-rs>.

Go

Для Go рекомендуется проект `Go-GL`: <https://github.com/go-gl>. В частности, понадобятся:

- Биндинги OpenGL: <https://github.com/go-gl/gl>
- Биндинги GLFW: <https://github.com/go-gl/glfw>
- Математика: <https://github.com/go-gl/mathgl>

JavaScript

Пожалуй, с JavaScript ситуация обстоит проще всего: здесь вообще не нужны никакие библиотеки, достаточно относительно современного браузера с поддержкой стандарта WebGL: <https://get.webgl.org/>. Для математики, правда, все-таки потребуется стороннее решение — рекомендуется взять `glMatrix`: <https://glmatrix.net/>. Однако, **обратите внимание**: стандарт WebGL основывается не на обычном OpenGL, а на OpenGL ES (OpenGL for Embedded Systems). Это влечет за собой некоторые отличия как в API, так и в коде шейдеров. Отличия не критические и для лаб не фатальные, но код с практик 1:1 перенести не получится, придется немного разбираться.

Приложение А. Исходный код программы

```
#include <iostream>
#include <GL/glew.h>
#include <GLFW/glfw3.h>

using namespace std;

GLFWwindow *g_window;

void init()
{
    // Set initial color of color buffer to blue.
    glClearColor(0.0f, 0.0f, 1.0f, 1.0f);
}

void reshape(GLFWwindow *window, int width, int height)
{
}

void draw()
{
    // Clear color buffer.
    glClear(GL_COLOR_BUFFER_BIT);
}
```

```

void cleanup()
{
}

bool initOpenGL()
{
    // Initialize GLFW functions.
    if (!glfwInit())
    {
        cout << "Failed to initialize GLFW" << endl;
        return false;
    }

    // Request OpenGL 3.3 without obsoleted functions.
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    // Create window.
    g_window = glfwCreateWindow(800, 600, "OpenGL Test", NULL, NULL);
    if (g_window == NULL)
    {
        cout << "Failed to open GLFW window" << endl;
        glfwTerminate();
        return false;
    }

    // Initialize OpenGL context with.
    glfwMakeContextCurrent(g_window);

    // Set internal GLEW variable to activate OpenGL core profile.
    glewExperimental = true;

    // Initialize GLEW functions.
    if (glewInit() != GLEW_OK)
    {
        cout << "Failed to initialize GLEW" << endl;
        return false;
    }

    // Ensure we can capture the escape key being pressed.
    glfwSetInputMode(g_window, GLFW_STICKY_KEYS, GL_TRUE);

    // Set callback for framebuffer resizing event.
    glfwSetFramebufferSizeCallback(g_window, reshape);

    return true;
}

void tearDownOpenGL()
{
    // Terminate GLFW.
    glfwTerminate();
}

int main()
{

```



```

// Initialize OpenGL
if (!initOpenGL())
    return -1;

// Initialize graphical resources.
init();

// Main loop until window closed or escape pressed.
while (glfwGetKey(g_window, GLFW_KEY_ESCAPE) != GLFW_PRESS &&
glfwWindowShouldClose(g_window) == 0)
{
    // Draw scene.
    draw();

    // Swap buffers.
    glfwSwapBuffers(g_window);
    // Poll window events.
    glfwPollEvents();
}

// Cleanup graphical resources.
cleanup();

// Tear down OpenGL.
tearDownOpenGL();

return 0;
}

```

Приложение Б. Исходный код отладочной функции

```

#include <stdio>

#include <GL/glew.h>

// В основной программе необходимо добавить строки:
// glfwWindowHint(GLFW_OPENGL_DEBUG_CONTEXT, true);
// ...
// glEnable(GL_DEBUG_OUTPUT);
// glEnable(GL_DEBUG_OUTPUT_SYNCHRONOUS); // Необязательно, но позволяет точнее
//                                           // поймать место возникновения ошибки
// glDebugMessageCallback(MessageCallback, nullptr);
// glDebugMessageControl(GL_DONT_CARE, GL_DONT_CARE, GL_DONT_CARE,
//                       0, nullptr, GL_TRUE);

void GLAPIENTRY
MessageCallback( GLenum source,
                GLenum type,
                GLuint id,
                GLenum severity,
                GLsizei length,
                const GLchar* message,
                const void* userParam )
{
    (void)source;
    (void)id;
    (void)length;
    (void)userParam;

    char *s_source, *s_type, *s_id, *s_severity;

```

```

switch (source) {
    case GL_DEBUG_SOURCE_API:
        s_source = "API";
        break;
    case GL_DEBUG_SOURCE_WINDOW_SYSTEM:
        s_source = "WINDOW_SYSTEM";
        break;
    case GL_DEBUG_SOURCE_SHADER_COMPILER:
        s_source = "SHADER_COMPILER";
        break;
    case GL_DEBUG_SOURCE_THIRD_PARTY:
        s_source = "THIRD_PARTY";
        break;
    case GL_DEBUG_SOURCE_APPLICATION:
        s_source = "APPLICATION";
        break;
    case GL_DEBUG_SOURCE_OTHER:
        s_source = "OTHER";
        break;
    default:
        s_source = "UNKNOWN";
        break;
}

```

```

switch (type) {
    case GL_DEBUG_TYPE_ERROR:
        s_type = "ERROR";
        break;
    case GL_DEBUG_TYPE_DEPRECATED_BEHAVIOR:
        s_type = "DEPRECATED_BEHAVIOR";
        break;
    case GL_DEBUG_TYPE_UNDEFINED_BEHAVIOR:
        s_type = "UNDEFINED_BEHAVIOR";
        break;
    case GL_DEBUG_TYPE_PORTABILITY:
        s_type = "PORTABILITY";
        break;
    case GL_DEBUG_TYPE_PERFORMANCE:
        s_type = "PERFORMANCE";
        break;
    case GL_DEBUG_TYPE_OTHER:
        s_type = "OTHER";
        break;
    case GL_DEBUG_TYPE_MARKER:
        s_type = "MARKER";
        break;
    default:
        s_type = "UNDEFINED";
        break;
}

```

```

switch (id) {
    case GL_BUFFER:
        s_id = "BUFFER";
        break;
    case GL_SHADER:
        s_id = "SHADER";
        break;
    case GL_PROGRAM:

```

```

        s_id = "PROGRAM";
        break;
    case GL_QUERY:
        s_id = "QUERY";
        break;
    case GL_PROGRAM_PIPELINE:
        s_id = "PROGRAM_PIPELINE";
        break;
    case GL_SAMPLER:
        s_id = "SAMPLER";
        break;
    default:
        s_id = "UNKNOWN";
        break;
}

switch (severity) {
    case GL_DEBUG_SEVERITY_HIGH:
        s_severity = "HIGH";
        break;
    case GL_DEBUG_SEVERITY_MEDIUM:
        s_severity = "MEDIUM";
        break;
    case GL_DEBUG_SEVERITY_LOW:
        s_severity = "LOW";
        break;
    case GL_DEBUG_SEVERITY_NOTIFICATION:
        s_severity = "NOTIFICATION";
        break;
    default:
        s_severity = "UNKNOWN";
        break;
}

fprintf( stderr, "GL CALLBACK: %s source = %s (0x%x), type = %s (0x%x),
severity = %s (0x%x), id = %s (0x%x), message = %s\n",
        ( type == GL_DEBUG_TYPE_ERROR ? "** GL ERROR **" : "" ),
        s_source, source,
        s_type, type,
        s_severity, severity,
        s_id, id,
        message );
}

```